

Soft Computing Mini Project

Master of Technology in CSE | Technical Report

1. Executive Summary

This document presents the complete lifecycle of the "Soft Computing Mini Project", focusing on building an Industrial IoT Machine Fault Classifier using a Single-Layer Perceptron from scratch. The project demonstrates the foundational principles of Artificial Neural Networks (ANNs) without relying on high-level machine learning libraries like Scikit-Learn, TensorFlow, or Pandas. By constructing the neuron, the activation function, and the backpropagation logic manually using core mathematics, it showcases a deep understanding of soft computing techniques.

2. The Problem Statement

The objective is to classify a factory machine's operational status into two distinct categories based on sensory data:

- Label 0: Healthy Machine (Normal operation) - No action required.
- Label 1: Faulty Machine (Needs immediate maintenance) - Trigger Alert.

To ensure visual transparency and mathematical simplicity, the network uses two input features:

- x_1 = Temperature Severity (0-10 scale): Measures how much the machine's temperature exceeds normal operating baselines.
- x_2 = Vibration Intensity (0-10 scale): Measures the intensity of irregular physical rattling or mechanical shaking.

3. The Model: Single-Layer Perceptron

The core of this project is a Single-Layer Perceptron, representing a single artificial neuron. It processes inputs through a weighted sum and applies a non-linear activation function to render a decision.

Step A: Feedforward (Weighted Sum)

$$z = w_1 * x_1 + w_2 * x_2 + b$$

Here, w_1 and w_2 represent the *synaptic weights*, determining the significance of each sensor reading. The variable b represents the *bias*, which acts as an intrinsic threshold shifting the neuron's activation sensitivity.

Step B: Neuron Activation

$$\text{output} = 1 / (1 + e^{(-z)})$$

The Sigmoid activation function is employed to compress the raw weighted sum into a continuous probability value bounded between 0 and 1, representing the network's confidence of a fault occurrence.

Step C: Output Threshold

A deterministic threshold converts the probability into a binary classification:

- If $\text{output} \geq 0.5$ -> The neuron fires, predicting Faulty (1).
- If $\text{output} < 0.5$ -> The neuron remains dormant, predicting Healthy (0).

4. The Learning Algorithm: Backpropagation

The perceptron initializes synaptic weights and bias with small random numbers. Optimization occurs over numerous training cycles (epochs) utilizing gradient descent.

Step-by-Step Training Pipeline:

1. Forward Pass: Propagate the input features through the network to generate a predicted output.
2. Error Calculation: Quantify the prediction error using Binary Cross-Entropy loss against the true label.
3. Backpropagation: Compute the gradient (the derivative of the error with respect to each weight) to evaluate each weight's contribution to the total error.
4. Weight Optimization: Adjust the synaptic weights by a small factor (the *learning rate*) in the direction opposite to the gradient.
5. Iteration: Repeat the process for 2,000 epochs until the loss converges and the decision boundary stabilizes.

5. Project Rationale & Real-World Impact

Why I Chose This Topic

- Real-World Relevance: Predictive maintenance in Industrial IoT (IIoT) is a crucial challenge. Classifying machine faults from sensor data demonstrates a high-value industrial application compared to overused academic datasets.
- Core Soft Computing Alignment: Stripping away black-box frameworks proves a fundamental understanding of how biological inspiration translates into mathematical models.
- Visual Transparency: Restricting inputs to two features allows the decision boundary to be plotted on a 2D plane, visually proving *how* the network learns.

Importance and Future Scope

In manufacturing, reactive maintenance is dangerous and costly. Models like this facilitate predictive maintenance, detecting early warning signs and preventing unplanned downtime.

Looking forward, this architecture provides a basis for:

- Multi-Layer Perceptrons (MLPs): Adding hidden layers to handle non-linear machine failures.
- Neuro-Fuzzy Integration: Evolving into an Adaptive Neuro-Fuzzy Inference System (ANFIS) to handle noisy sensor data by combining fuzzy logic's tolerance for uncertainty with neural learning.
- Edge AI Deployment: Because this model is mathematically lightweight, it can be deployed directly onto edge microcontrollers in factory motors for real-time monitoring.

6. Appendix: Implementation Code

(Filename: perceptron_classifier_scratch.py)

```
import math
```

```

import random

# Dataset [Temperature, Vibration]
X_train = [[1.0, 2.0], [2.0, 1.5], [8.0, 9.0], [9.0, 7.5]] # Truncated for
display
y_train = [0, 0, 1, 1]

def sigmoid(z):
    z = max(-500, min(500, z))
    return 1.0 / (1.0 + math.exp(-z))

# Initialization
w1, w2, b = random.uniform(-1, 1), random.uniform(-1, 1), random.uniform(-1, 1)
learning_rate = 0.1

# Training
for epoch in range(2000):
    for i in range(len(X_train)):
        x1, x2 = X_train[i]
        y_true = y_train[i]

        z = (w1 * x1) + (w2 * x2) + b
        y_pred = sigmoid(z)

        error = y_pred - y_true
        w1 -= learning_rate * error * x1
        w2 -= learning_rate * error * x2
        b -= learning_rate * error

```